

Préparation de l'environnement **Supabase** pour ADMINA-RH

Document explicatif détaillé de la Phase 0 : création des schémas PostgreSQL admina et admina_rh, configuration des permissions, installation des extensions, table des tenants, et mise en place du système d'isolation multi-tenant par Row Level Security.

PROJET

ADMINA-RH — Schémas admina / admina_rh

TECHNOLOGIE

Supabase • PostgreSQL 17.6 • eu-west-1 • Plan Pro

ETAPES

5 étapes • 4 extensions • 1 table tenants • RLS

Table des matieres

1. Introduction : qu'est-ce que la Phase 0 ?	2
1.1 Definition et objectif	2
1.2 Pourquoi cette phase est critique	2
1.3 Contexte technique Supabase	2
2. Architecture des schemas PostgreSQL	3
2.1 Le schema parent : admina	3
2.2 Le schema enfant : admina_rh	3
2.3 Conventions de nommage	3
2.4 Schema public vs schema admina	4
3. Les 5 etapes detaillees de la Phase 0	4
3.1 Etape 1 : Creation des schemas	4
3.2 Etape 2 : Configuration des permissions	5
3.3 Etape 3 : Extensions PostgreSQL	5
3.4 Etape 4 : Table des tenants	6
3.5 Etape 5 : Configuration multi-tenant (RLS)	7
4. Modele de securite multi-tenant	7
4.1 Principe du Row Level Security	7
4.2 Les 5 roles du systeme ADMINA	8
4.3 Flux d'authentification JWT	8
5. Verification et validation de la Phase 0	9
5.1 Script de verification automatique	9
5.2 Tests d'isolation multi-tenant	9
5.3 Checklist de validation	10
6. Ce qui suit la Phase 0	10
6.1 Vue d'ensemble des phases suivantes	10
6.2 Prerequis pour demarrer la Phase 0	11

1. Introduction : qu'est-ce que la Phase 0 ?

1.1 Definition et objectif

La Phase 0 est l'etape preparatoire fondamentale qui precede toute creation de tables dans la base de donnees Supabase. Elle consiste a preparer le terrain en creant la structure d'accueil des donnees : les schemas PostgreSQL, les extensions necessaires, la table des tenants, les roles de securite, et les mecanismes d'isolation multi-tenant. Sans cette phase, il est impossible de creer les tables du Departement 1 ou de tout autre domaine de maniere securisee. La Phase 0 est executee une seule fois au debut du projet et ne necessite pas de modification ulterieure, sauf ajout de nouveaux tenants.

L'objectif principal de la Phase 0 est de poser les fondations techniques qui permettront a l'ensemble des 31 domaines RH de fonctionner de maniere isolee et securisee. Chaque entreprise cliente (tenant) ne doit voir que ses propres donnees, et chaque role d'utilisateur ne doit acceder qu'aux operations autorisees. Cette isolation est garantie au niveau de la base de donnees par le Row Level Security (RLS) de PostgreSQL, et non au niveau applicatif, ce qui constitue une couche de securite supplementaire incontournable.

1.2 Pourquoi cette phase est critique

La Phase 0 est critique pour trois raisons majeures. Premiere raison : la **securite**. Une erreur dans la configuration des schemas ou des permissions peut exposer les donnees d'un tenant a un autre, ce qui serait une violation grave de la confidentialite. Deuxieme raison : l'**irreversibilite partielle**. Bien que les schemas et les tables puissent etre modifies ulterieurement, certaines decisions structurelles (comme le prefixe de nommage des tables ou la strategie d'isolation multi-tenant) sont difficiles a changer une fois que les 620 tables des 31 domaines sont en place. Troisieme raison : la **dependance**. Toutes les phases suivantes (Phase 1 pour les ENUMs, Phase 2 pour les tables, etc.) dependent de la Phase 0. Si les schemas ou les permissions sont mal configures, chaque migration subsequente echouera.

1.3 Contexte technique Supabase

L'environnement cible est un projet Supabase sur le plan Pro, heberge dans la region eu-west-1, avec PostgreSQL 17.6. Supabase fournit une API REST auto-generee (PostgREST), un systeme d'authentification (GoTrue), un stockage de fichiers (Storage), et un systeme de notifications en temps reel (Realtime). L'identification du projet est la reference `aywwakllgvfoqlpowzqf`. L'accès administratif se fait via le Management API (token `sbp_...`) ou directement via une connexion PostgreSQL avec le mot de passe de la base de donnees. Pour la Phase 0, nous utiliserons la connexion directe PostgreSQL car les operations de creation de schemas et d'extensions ne sont pas disponibles via l'API REST.

Parametre	Valeur
Projet Supabase	aywwakllgvfoqlpowzqf
Region	eu-west-1 (Irlande)
Plan	Pro
PostgreSQL	17.6

Parametre	Valeur
Schema parent	admina
Schema enfant	admina_rh
Authentification	GoTrue (JWT)
Isolation	Row Level Security (RLS)
Nombre de tenants prevu	Multi-tenant illimite
Nombre de domaines	31 (D1 a D31)

Tableau 1 : Parametres techniques du projet Supabase

2. Architecture des schemas PostgreSQL

2.1 Le schema parent : admina

Le schema **admina** est le conteneur parent de tous les sous-projets de l'organisation. Il joue le role de 'dossier racine' dans lequel seront crees les schemas enfants (admina_rh, admina_comptable, admina_commerce, etc.). Physiquement, dans PostgreSQL, un schema est un namespace qui regroupe des tables, des types, des fonctions et d'autres objets de base de donnees. Le schema admina contiendra les elements suivants : la table des tenants (qui liste toutes les entreprises clientes), les types ENUM partages entre plusieurs sous-projets (par exemple, type_contrat, civilite, genre), les fonctions utilitaires (trigger de mise a jour de updated_at, fonction de recherche full-text), et les politiques RLS par default qui s'appliqueront a tous les schemas enfants.

Le schema admina est cree une seule fois avec la commande `CREATE SCHEMA IF NOT EXISTS admina`. Le `IF NOT EXISTS` garantit que la commande est idempotente : elle peut etre relancee sans erreur si le schema existe deja. Le proprietaire du schema est le role **postgres** (superuser), qui est le role utilise par les migrations SQL. Les permissions d'accès au schema sont accordees explicitement aux roles **anon** et **authenticated**, qui sont les roles utilises par l'API REST de Supabase.

2.2 Le schema enfant : admina_rh

Le schema **admina_rh** est le namespace dedie a l'ensemble du projet ADMINA-RH. Il contiendra les 620 tables des 31 domaines (environ 20 tables par domaine en moyenne), les types ENUM specifiques a chaque domaine, les vues materialisees pour les tableaux de bord, les triggers metier, et les fonctions de calcul. Toutes les tables du schema admina_rh incluront la colonne **tenant_id** pour garantir l'isolation multi-tenant via les politiques RLS.

Le schema admina_rh est cree avec la commande `CREATE SCHEMA IF NOT EXISTS admina_rh`. Il est important de noter que dans PostgreSQL, les schemas sont des objets de premier niveau : il n'y a pas de relation de 'parent-enfant' structurelle entre les schemas. L'organisation hierarchique (admina contient admina_rh) est une **convention de nommage et d'organisation**, pas une contrainte technique. Cela signifie qu'un objet du schema admina_rh peut reference un objet du schema admina sans restriction, et reciproquement. Cette flexibilite est un avantage pour les interconnexions entre domaines, mais necessite une discipline rigoureuse dans la gestion des permissions.

2.3 Conventions de nommage

Les conventions de nommage sont essentielles pour maintenir la coherence et la lisibilite de la base de donnees a mesure qu'elle grandit. Chaque table du schema `admina_rh` sera prefixee par le numero de domaine suivi d'un underscore. Par exemple, les tables du Departement 1 auront le prefixe `d1_`, celles du Departement 2 le prefixe `d2_`, et ainsi de suite. Les types ENUM suivent la meme logique : les types specifiques a un domaine sont prefixes (par exemple `d1_statut_demande`), tandis que les types partages entre plusieurs domaines n'ont pas de prefixe (par exemple `type_contrat`). Les fonctions et triggers sont prefixes par le nom du schema pour eviter les collisions.

Type d'objet	Convention	Exemple
Table domaine	<code>d{N}_{nom_table}</code>	<code>d1_demandes_recrutement</code>
ENUM specifique	<code>d{N}_{nom_enum}</code>	<code>d1_statut_demande</code>
ENUM partage	<code>{nom_enum}</code>	<code>type_contrat</code> , <code>civilite</code>
Fonction	<code>{schema}_{nom}</code>	<code>admina_rh.trigger_updated_at</code>
Trigger	<code>trg_{table}_{action}</code>	<code>trg_d1_demandes_updated</code>
Index	<code>idx_{table}_{colonnes}</code>	<code>idx_d1_demandes_tenant_statut</code>
Vue	<code>vw_{nom_descriptif}</code>	<code>vw_d1_tableau_bord_recrutement</code>
Politique RLS	<code>{action}_{table}_{role}</code>	<code>sel_demandes_admin_tenant</code>

Tableau 2 : Conventions de nommage des objets de base de donnees

2.4 Schema public vs schema admina

Par default, Supabase place les tables dans le schema `public`. C'est le schema par default de PostgreSQL et celui que l'API REST utilise si aucun schema n'est specifie. Cependant, pour le projet ADMINA, nous n'utiliserons pas le schema `public`. Toutes les tables seront dans le schema `admina` (pour les objets partages) ou `admina_rh` (pour les tables des domaines RH). Le schema `public` restera vide. Cette decision a plusieurs avantages : elle separe clairement les donnees applicatives des objets systeme Supabase (qui restent dans `public`), elle facilite la gestion des permissions (`GRANT` sur des schemas entiers), et elle prepare le terrain pour les futurs schemas `admina_comptable` et `admina_commerce` qui suivront la meme convention.

3. Les 5 etapes detaillees de la Phase 0

3.1 Etape 1 : Creation des schemas

La premiere etape consiste a creer les deux schemas PostgreSQL : **admina** (parent) et **admina_rh** (enfant). Cette operation est simple en soi mais a des implications profondes car elle definit le namespace dans lequel toutes les futures tables seront creees. La commande SQL utilise `CREATE SCHEMA IF NOT EXISTS` pour garantir l'idempotence : si le schema existe deja (par exemple lors d'une re-execution de la migration), la commande ne produit pas d'erreur et ne modifie pas le schema existant.

```
-- Etape 1 : Creation des schemas
CREATE SCHEMA IF NOT EXISTS admina;
```

```
CREATE SCHEMA IF NOT EXISTS admina_rh;

-- Verification
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN ('admina', 'admina_rh')
ORDER BY schema_name;
```

Après execution, la verification confirme que les deux schemas sont bien presents dans la base de donnees. Le schema public reste intact (il contient les objets systeme Supabase), et les schemas admina et admina_rh sont vides, prêts a recevoir les objets des phases suivantes. Il est important de noter que la suppression d'un schema (DROP SCHEMA ... CASCADE) supprime tous les objets qu'il contient, y compris les tables, les types et les fonctions. Cette operation est irreversible et ne doit etre executee qu'en environnement de developpement.

3.2 Etape 2 : Configuration des permissions

La deuxieme etape configure les permissions d'accès aux schemas pour les deux roles utilises par l'API REST de Supabase : **anon** (accès non authentifié, en lecture seule) et **authenticated** (accès authentifié, lecture et ecriture). Par default, seuls le superuser et le propriétaire du schema ont accès a un nouveau schema. Les autres roles ne peuvent ni voir ni utiliser les objets du schema tant que des permissions explicites ne sont pas accordees.

```
-- Etape 2 : Permissions sur les schemas
-- Le role 'anon' peut lister et lire les objets
GRANT USAGE ON SCHEMA admina TO anon;
GRANT USAGE ON SCHEMA admina_rh TO anon;
GRANT SELECT ON ALL TABLES IN SCHEMA admina TO anon;
GRANT SELECT ON ALL TABLES IN SCHEMA admina_rh TO anon;

-- Le role 'authenticated' peut lister, lire, ecrire
GRANT USAGE ON SCHEMA admina TO authenticated;
GRANT USAGE ON SCHEMA admina_rh TO authenticated;
GRANT ALL ON ALL TABLES IN SCHEMA admina TO authenticated;
GRANT ALL ON ALL TABLES IN SCHEMA admina_rh TO authenticated;

-- Permettre aux roles de creer des objets (pour les migrations)
GRANT CREATE ON SCHEMA admina TO authenticated;
GRANT CREATE ON SCHEMA admina_rh TO authenticated;
```

La permission **USAGE** sur un schema permet a un role de lister les objets du schema (tables, fonctions, types) et de les reference dans des requetes SQL. Sans cette permission, meme si un role a le droit **SELECT** sur une table, il ne peut pas y acceder car il ne peut pas 'voir' le schema. La permission **CREATE** permet de creer de nouveaux objets dans le schema. Elle est accordee au role **authenticated** pour permettre les futures migrations via l'API REST. En production, cette permission pourrait etre retiree et reservee uniquement au role superuser utilise par les scripts de migration.

3.3 Etape 3 : Extensions PostgreSQL

La troisieme etape installe les extensions PostgreSQL necessaires au fonctionnement du projet ADMINA-RH. PostgreSQL supporte un systeme d'extensions modulaires qui ajoute des fonctionnalites supplementaires a la base de donnees. Pour le projet ADMINA-RH, quatre extensions sont requises. L'extension **pgcrypto** fournit des fonctions cryptographiques, notamment **gen_random_uuid()** utilisee pour generer des UUIDs comme cles

primaires. L'extension `uuid-oss` fournit des fonctions complementaires de generation et de manipulation d'UUIDs. L'extension `pg_stat_statements` active le suivi des statistiques d'execution des requetes, utile pour le monitoring des performances. L'extension `pg_trgm` fournit la recherche textuelle floue (trigram matching), utilisee pour la barre de recherche globale de candidats.

```
-- Etape 3 : Extensions PostgreSQL
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Extension	Fonctions cles	Usage dans ADMINA-RH
pgcrypto	gen_random_uuid(), digest(), encrypt()	Cles primaires UUID, hashage mots de passe
uuid-oss	uuid_generate_v4(), uuid_nil()	Generation UUID alternative, valeurs nulles
pg_stat_statements	pg_stat_statements_reset()	Monitoring des requetes lentes
pg_trgm	similarity(), show_trgm(), % operator	Recherche floue noms, emails, postes

Tableau 3 : Extensions PostgreSQL requises et leurs usages

3.4 Etape 4 : Table des tenants

La quatrieme etape cree la table des tenants dans le schema admina. Cette table est le coeur du systeme multi-tenant : chaque entreprise cliente est un tenant, et chaque table des domaines RH reference cette table via la colonne `tenant_id`. La table des tenants contient les informations d'identification de l'entreprise (nom, slug, secteur), les informations de contact, les parametres de configuration (devise, fuseau horaire, langue), et les indicateurs d'etat (actif/inactif, date de creation, date d'expiration de l'abonnement).

```
-- Etape 4 : Table des tenants
CREATE TABLE admina.tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(200) NOT NULL,
  slug VARCHAR(100) UNIQUE NOT NULL,
  sector VARCHAR(100),
  country VARCHAR(100) DEFAULT 'Cameroun',
  city VARCHAR(100),
  contact_email VARCHAR(255),
  contact_phone VARCHAR(30),
  is_active BOOLEAN DEFAULT true,
  plan VARCHAR(50) DEFAULT 'pro',
  settings JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Index pour la recherche rapide par slug
CREATE UNIQUE INDEX idx_tenants_slug
ON admina.tenants(slug);
```

```
-- Donnee de test
INSERT INTO admina.tenants (name, slug, sector, contact_email)
VALUES ('Hotel Premium Douala', 'hotel-premium-dla', 'Hotellerie',
'drh@hotelpremium.cm');
```

La colonne **slug** est un identifiant unique lisible par l'humain, utilise dans les URLs et les noms de fichiers. La colonne **settings** est un champ JSONB qui stocke les parametres specifiques a chaque tenant (devise FCFA/XAF, fuseau horaire Africa/Douala, langue fr, taux d'absence maximal, etc.). Ce format flexible permet d'ajouter des parametres sans modifier la structure de la table. Le champ **plan** stocke le type d'abonnement du tenant (free, pro, entreprise) et sera utilise par les futures politiques d'autorisation pour limiter les fonctionnalites selon le plan.

3.5 Etape 5 : Configuration multi-tenant (RLS)

La cinquieme etape configure le mecanisme d'isolation multi-tenant via le Row Level Security. Cette etape est la plus complexe de la Phase 0 car elle implique plusieurs composants interconnectes : un database hook qui extrait le **tenant_id** du JWT et l'injecte dans un parametre de session PostgreSQL, une fonction helper qui lit ce parametre, et une politique RLS par default qui s'applique a toutes les futures tables du schema **admina_rh**.

Le mecanisme fonctionne de la maniere suivante. Lorsqu'un utilisateur se connecte via l'API Supabase, son JWT contient une revendication (claim) **app.tenant_id** qui identifie son entreprise. Avant chaque requete, un **database hook** (configure dans le tableau **supabase.auth.hooks**) extrait cette valeur et l'injecte dans un parametre de session PostgreSQL via **SET LOCAL app.tenant_id**. Ensuite, chaque politique RLS lit ce parametre via **current_setting('app.tenant_id')::uuid** et filtre les lignes en consequence. Ce mecanisme garantit que meme si un utilisateur modifie manuellement une requete pour tenter d'accéder aux donnees d'un autre tenant, la politique RLS bloquera l'accès.

```
-- Etape 5 : Fonction helper multi-tenant
CREATE OR REPLACE FUNCTION admina.current_tenant_id()
RETURNS UUID AS $$
DECLARE
tid UUID;
BEGIN
tid := NULLIF(
current_setting('app.tenant_id', true), '')::UUID;
IF tid IS NULL THEN
RAISE EXCEPTION 'Tenant ID non defini dans la session';
END IF;
RETURN tid;
END;
$$ LANGUAGE plpgsql STABLE;
```

4. Modele de securite multi-tenant

4.1 Principe du Row Level Security

Le Row Level Security (RLS) est une fonctionnalite native de PostgreSQL (depuis la version 9.5) qui permet de definir des regles d'accès au niveau de la ligne (et non de la table). Contrairement aux permissions **GRANT/REVOKE** qui s'appliquent a une table entiere, les politiques RLS filtrent les lignes retournees par une

requete en fonction de conditions definies. Par exemple, une politique RLS peut garantir qu'un utilisateur ne voit que les lignes dont la colonne `tenant_id` correspond a son entreprise.

Dans Supabase, le RLS est le mecanisme de securite recommande pour les applications multi-tenant. L'API REST de Supabase (PostgREST) applique automatiquement les politiques RLS a chaque requete. Lorsque le RLS est active sur une table, seules les lignes qui satisfont les politiques sont retournees par les requetes `SELECT`, et seules les lignes qui satisfont les politiques `WITH CHECK` peuvent etre inserees ou modifiees. Si aucune politique n'est definie pour une operation (`SELECT`, `INSERT`, `UPDATE`, `DELETE`), cette operation est interdite par default. C'est un mecanisme 'deny by default' qui est plus securise que le 'allow by default' des permissions `GRANT`.

4.2 Les 5 roles du systeme ADMINA

Le systeme ADMINA-RH definit 5 roles qui determinent les permissions d'accès de chaque utilisateur. Ces roles sont stockes dans le JWT de l'utilisateur sous forme de revendication `app.role` et sont lus par les politiques RLS pour chaque requete. Le role **admin_tenant** est le superutilisateur du tenant : il peut lire, creer, modifier et supprimer toutes les donnees de son entreprise. Ce role est reserve au dirigeant ou au DAF. Le role **drh** est le directeur des ressources humaines : il a un acces complet a toutes les donnees RH mais ne peut pas modifier les parametres systeme du tenant. Le role **manager** est un responsable de departement : il peut creer des demandes de recrutement et consulter les donnees de son departement uniquement. Le role **rh_agent** est un agent RH : il peut gerer les operations quotidiennes de recrutement mais n'a pas accès aux donnees sensibles (salaires, evaluations). Le role **employee** est un employe standard : il a un acces en lecture seule a ses propres donnees (profil, contrat, conges).

Role	Profil	Portee	Permissions typiques
admin_tenant	Dirigeant / DAF	Tout le tenant	CRUD complet sur toutes les tables
drh	Directeur RH	Tout le tenant	CRUD sur toutes les tables RH
manager	Resp. departement	Son departement	CR demandes, R donnees de son dept.
rh_agent	Agent RH	Operations courantes	CRUD recrutement, R sur paie/perf.
employee	Employe	Ses propres donnees	R profil, contrat, conges, notes

Tableau 4 : Les 5 roles RBAC du systeme ADMINA-RH

4.3 Flux d'authentification JWT

Le flux d'authentification dans ADMINA-RH fonctionne en plusieurs etapes coordonnees. Premierement, l'utilisateur se connecte via l'API GoTrue de Supabase avec son email et mot de passe. Deuxiemement, GoTrue valide les identifiants et genere un JWT contenant les revendications personnalisées : `app.tenant_id` (identifiant de l'entreprise), `app.role` (role de l'utilisateur parmi les 5 definis), `app.department` (departement de l'utilisateur, pour le filtre manager), et `app.user_name` (nom complet, pour les traces d'audit). Troisiemement, le JWT est envoye au client qui le stocke localement et l'inclut dans le header `Authorization: Bearer {token}` de chaque requete API. Quatriemement, a chaque requete, PostgREST extrait le JWT, le valide, et transmet les revendications a PostgreSQL via des variables de session. Cinquiemement, les politiques RLS utilisent ces variables pour filtrer les donnees.

5. Verification et validation de la Phase 0

5.1 Script de verification automatique

Apres l'execution de chacune des 5 etapes de la Phase 0, un script de verification automatique doit etre execute pour confirmer que tous les objets ont ete correctement cree et que les permissions sont correctement configurees. Ce script execute une serie de requetes SQL de verification et affiche un rapport succinct indiquant le statut de chaque verification (OK ou ERREUR). Le script ci-dessous peut etre execute via **psql** ou via l'editeur SQL de Supabase.

```
-- Script de verification Phase 0
-- 1. Schemas
SELECT 'Schemas' AS check,
CASE WHEN count(*) = 2 THEN 'OK'
ELSE 'ERREUR: attendu 2' END AS status
FROM information_schema.schemata
WHERE schema_name IN ('admina','admina_rh');

-- 2. Extensions
SELECT 'Extensions' AS check,
CASE WHEN count(*) >= 4 THEN 'OK'
ELSE 'ERREUR: ' || count(*) || '/4' END AS status
FROM pg_extension
WHERE extname IN ('pgcrypto','uuid-ossf',
'pg_stat_statements','pg_trgm');

-- 3. Table tenants
SELECT 'Table tenants' AS check,
CASE WHEN EXISTS (
SELECT 1 FROM information_schema.tables
WHERE table_schema='admina' AND table_name='tenants'
) THEN 'OK' ELSE 'ERREUR' END AS status;

-- 4. Fonction helper
SELECT 'Fonction current_tenant_id' AS check,
CASE WHEN EXISTS (
SELECT 1 FROM pg_proc p
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname='admina'
AND p.proname='current_tenant_id'
) THEN 'OK' ELSE 'ERREUR' END AS status;
```

5.2 Tests d'isolation multi-tenant

La verification la plus importante de la Phase 0 est le test d'isolation multi-tenant. Ce test consiste a creer deux tenants de test, a inserer des donnees dans chaque tenant, puis a verifier que chaque tenant ne peut acceder qu'a ses propres donnees. Le test est execute en simulant deux sessions utilisateur differentes, chacune avec un tenant_id different. Pour chaque session, on verifie que les requetes SELECT ne retournent que les lignes du tenant courant, que les requetes INSERT avec un tenant_id different sont rejetees, et que les requetes UPDATE sur les lignes d'un autre tenant sont rejetees. Ce test doit etre execute apres la Phase 2 (creation des tables) car il necessite au moins une table avec des politiques RLS actives.

5.3 Checklist de validation

La validation complete de la Phase 0 se resume a la checklist suivante, qui doit etre verifiee point par point avant de passer a la Phase 1. Chaque item est soit verifie automatiquement par le script de verification, soit manuellement par le developpeur via l'interface d'administration Supabase. Aucun item ne doit etre marque comme 'non verifie' avant de passer a la phase suivante, car chaque item est un pre-requis pour les phases ulterieures.

Item	Critere de validation	Methode
Schema admina	Existe et est accessible	Script SQL automatique
Schema admina_rh	Existe et est accessible	Script SQL automatique
Extension pgcrypto	gen_random_uuid() fonctionne	SELECT gen_random_uuid()
Extension pg_trgm	La fonction similarity() fonctionne	SELECT similarity('test','tset')
Table admina.tenants	Existe avec 14 colonnes	information_schema.columns
Index idx_tenants_slug	Existe et est UNIQUE	pg_indexes
Donnee de test	1 tenant insere	SELECT count(*) FROM admina.tenants
Fonction current_tenant_id()	Retourne un UUID	SELECT admina.current_tenant_id()
Permissions anon	USAGE sur admina et admina_rh	information_schema.role_schema_grants
Permissions authenticated	USAGE + CREATE sur les schemas	information_schema.role_schema_grants

Tableau 5 : Checklist de validation de la Phase 0

6. Ce qui suit la Phase 0

6.1 Vue d'ensemble des phases suivantes

Une fois la Phase 0 validee, le projet entre dans les phases de migration proprement dites. La Phase 1 cree les types ENUM (specifiques et partages) necessaires a l'ensemble des domaines. La Phase 2 cree les 17 tables du Departement 1 dans le schema admina_rh, avec toutes leurs contraintes (NOT NULL, UNIQUE, CHECK, FOREIGN KEY). La Phase 3 ajoute la couche logique : indexes pour les performances, triggers pour les mises a jour automatiques, et fonctions pour la recherche et les notifications. La Phase 4 active le Row Level Security sur chaque table et cree les politiques d'accès pour les 5 roles. Enfin, la Phase 5 injecte les donnees de test, valide l'isolation multi-tenant, verifie les performances et confirme la coherence des donnees migrees.

Phase	Nom	Duree estimee	Livable
0	Preparation de l'environnement	0.5 jour	Schemas, extensions, table tenants, RLS setup

Phase	Nom	Duree estimee	Livrable
1	Types ENUM et references	0.5 jour	39 types ENUM, tables de reference partagees
2	Tables du Departement 1	2 jours	17 tables SQL avec contraintes completes
3	Index, triggers, fonctions	1 jour	12 indexes, 17 triggers, 5 fonctions
4	Politiques RLS	1 jour	68 politiques de securite sur 17 tables
5	Donnees de test et validation	1 jour	500 lignes de test, rapport de validation

Tableau 6 : Roadmap des 6 phases de migration

6.2 Prerequis pour demarrer la Phase 0

Avant d'exécuter la Phase 0, trois prerequis doivent être satisfaits. Premièrement, les credentials Supabase doivent être valides et accessibles : le Management API token, le Service Role key, et le mot de passe de la base de données doivent permettre une connexion réussie au projet. Deuxièmement, l'environnement d'exécution (Python 3.12+, psycopg2 ou asyncpg, psycopg) doit être installé et fonctionnel. Troisièmement, le document de démarche du Département 1 (PDF généré précédemment) doit avoir été validé par l'équipe technique, car il définit les conventions de nommage et l'architecture qui seront appliquées dès la Phase 0. Une fois ces trois prerequis vérifiés, l'exécution de la Phase 0 peut commencer.